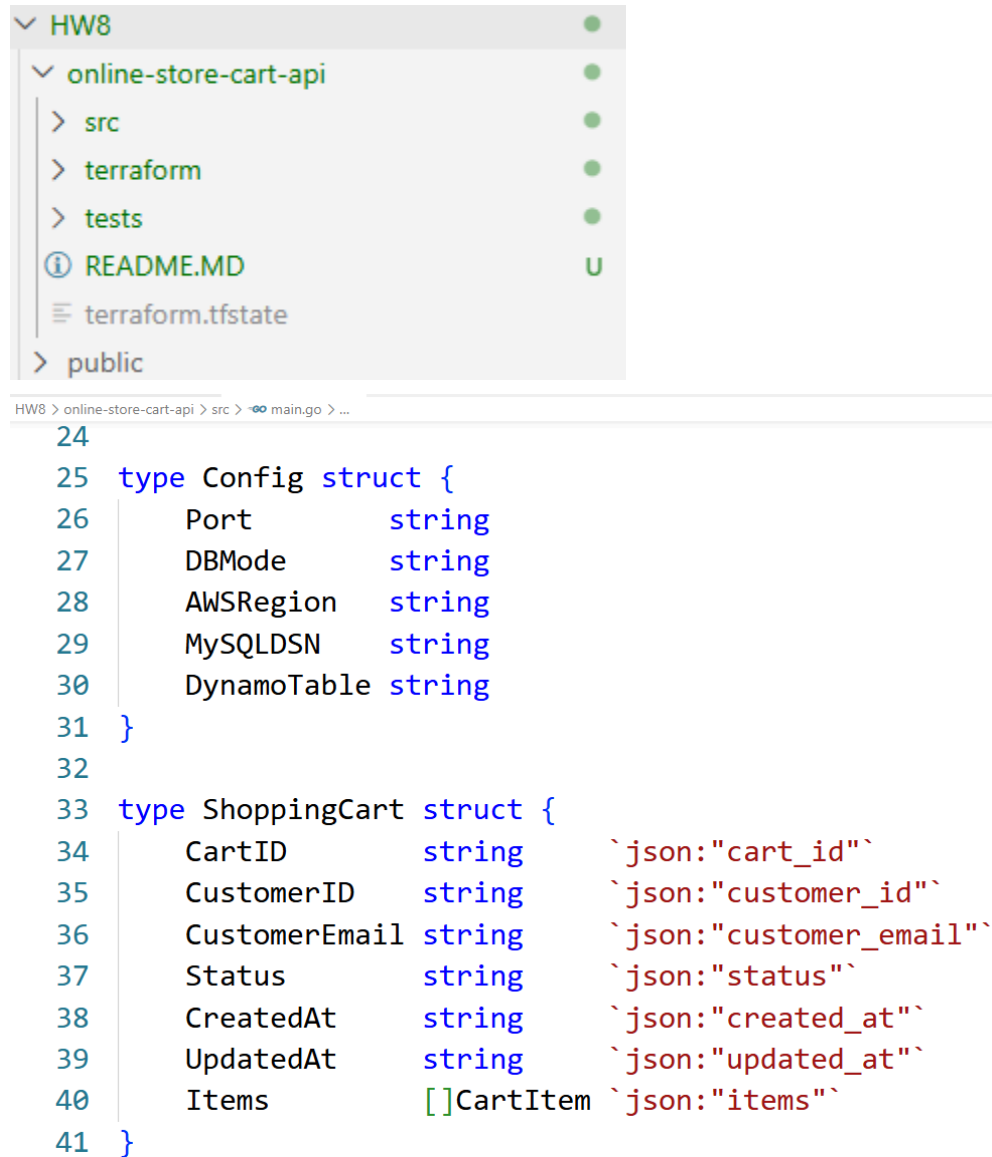


Homework 8 Report

Wilson Neira

STEP I: MySQL Integration for Relational Data



The screenshot shows a code editor with a file explorer on the left and Go code in the main area. The file explorer shows a directory structure for 'HW8' with subdirectories 'online-store-cart-api', 'src', 'terraform', 'tests', and 'public'. The 'online-store-cart-api' directory is expanded, showing 'src', 'terraform', 'tests', 'README.MD', and 'terraform.tfstate'. The main editor area shows Go code for a MySQL integration. The code defines a 'Config' struct with fields for 'Port', 'DBMode', 'AWSRegion', 'MySQLDSN', and 'DynamoTable'. It also defines a 'ShoppingCart' struct with fields for 'CartID', 'CustomerID', 'CustomerEmail', 'Status', 'CreatedAt', 'UpdatedAt', and 'Items'. The 'Items' field is a slice of 'CartItem' structs. The code is formatted with syntax highlighting and line numbers.

```
24
25 type Config struct {
26     Port      string
27     DBMode    string
28     AWSRegion string
29     MySQLDSN  string
30     DynamoTable string
31 }
32
33 type ShoppingCart struct {
34     CartID      string `json:"cart_id"`
35     CustomerID  string `json:"customer_id"`
36     CustomerEmail string `json:"customer_email"`
37     Status      string `json:"status"`
38     CreatedAt   string `json:"created_at"`
39     UpdatedAt   string `json:"updated_at"`
40     Items       []CartItem `json:"items"`
41 }
```

```

43 type CartItem struct {
44     ProductID string `json:"product_id"`
45     Name      string `json:"name"`
46     Quantity  int    `json:"quantity"`
47     UnitPrice float64 `json:"unit_price"`
48 }
49
50 type CreateCartRequest struct {
51     CustomerID   string `json:"customer_id"`
52     CustomerEmail string `json:"customer_email"`
53 }
54
55 type AddItemRequest struct {
56     ProductID string `json:"product_id"`
57     Name      string `json:"name"`
58     Quantity  int    `json:"quantity"`
59     UnitPrice float64 `json:"unit_price"`
60 }

```

Same Dockerfile as before

```

1 terraform {
2     required_providers {
3         aws = {
4             source = "hashicorp/aws"
5             version = "~> 6.0"
6         }
7     }
8 }
9
10 provider "aws" {
11     region = var.aws_region
12 }

```

HW8 > online-store-cart-api > terraform > 🐛 variables.tf

```
1 variable "aws_region" {
2 |   default = "us-east-1"
3 }
4
5 variable "project_name" {
6 |   default = "cartapi"
7 }
8
9 variable "container_image" {
10 |   type = string
11 }
12
13 variable "db_username" {
14 |   default = "cartuser"
15 }
16
17 variable "db_password" {
18 |   type      = string
```

HW8 > online-store-cart-api > terraform > 🐛 main.tf

```
1 resource "aws_vpc" "main" {
2 |   cidr_block      = "10.0.0.0/16"
3 |   enable_dns_support = true
4 |   enable_dns_hostnames = true
5 }
6
7 resource "aws_internet_gateway" "igw" {
8 |   vpc_id = aws_vpc.main.id
9 }
10
11 resource "aws_subnet" "public_1" {
12 |   vpc_id            = aws_vpc.main.id
13 |   cidr_block        = "10.0.1.0/24"
14 |   availability_zone = "${var.aws_region}a"
15 |   map_public_ip_on_launch = true
16 }
17
18 resource "aws_subnet" "public_2" {
19 |   vpc_id            = aws_vpc.main.id
```



```
25 resource "aws_db_instance" "mysql" {
26     identifier            = "${var.project_name}-mysql"
27     engine                = "mysql"
28     engine_version        = "8.0"
29     instance_class        = "db.t3.micro"
30     allocated_storage     = 20
31     max_allocated_storage = 20
32     db_name               = var.db_name
33     username              = var.db_username
34     password              = var.db_password
35     db_subnet_group_name  = aws_db_subnet_group.main.name
36     vpc_security_group_ids = [aws_security_group.rds.id]
37     publicly_accessible   = false
38     skip_final_snapshot   = true
39     deletion_protection   = false
40     multi_az              = false
41     backup_retention_period = 0
42 }
```

```
1 resource "aws_dynamodb_table" "shopping_carts" {
2     name           = "shopping_carts"
3     billing_mode   = "PAY_PER_REQUEST"
4     hash_key       = "cart_id"
5
6     attribute {
7         name = "cart_id"
8         type = "S"
9     }
10 }
```

HW8 > online-store-cart-api > terraform > 🐛 ecs.tf

```
1 data "aws_iam_role" "lab_role" {
2   name = "LabRole"
3 }
4
5 resource "aws_cloudwatch_log_group" "api" {
6   name           = "/ecs/cartapi"
7   retention_in_days = 7
8 }
9
10 resource "aws_ecs_task_definition" "api" {
11   family                = "cartapi"
12   requires_compatibilities = ["FARGATE"]
13   network_mode          = "awsvpc"
14   cpu                    = "256"
15   memory                 = "512"
16   execution_role_arn     = data.aws_iam_role.lab_role.arn
17   task_role_arn          = data.aws_iam_role.lab_role.arn
18
19   container_definitions = jsonencode([{"
```

HW8 > online-store-cart-api > terraform > 🐛 outputs.tf

```
1 output "alb_dns_name" {
2   value = aws_lb.main.dns_name
3 }
4
5 output "mysql_endpoint" {
6   value = aws_db_instance.mysql.address
7 }
8
9 output "dynamodb_table_name" {
10  value = aws_dynamodb_table.shopping_carts.name
11 }
12
13 output "ecs_cluster_name" {
14  value = aws_ecs_cluster.main.name
15 }
```

Database Schema Implementation

Why you chose your table structure?

I used two tables: `shopping_carts` and `shopping_cart_items`. I chose this structure because a shopping cart has one-to-many items, and a relational model makes that relationship explicit with a foreign key. This helps preserve data integrity and prevents orphaned cart items.

What indexes you added and why?

I made `cart_id` the primary key for `shopping_carts` and a composite primary key of (`cart_id`, `product_id`) for `shopping_cart_items`. I also added an index on `customer_id` so customer history queries are efficient. This matches the required access patterns of retrieving a cart by ID and finding carts by customer.

Connection pooling configuration decisions?

I configured connection pooling with a limited number of open and idle MySQL connections so the ECS task would not open unnecessary database sessions. This improved stability and matched the assignment requirement to use proper connection pooling.

Any trade-offs you considered?

The main trade-off with MySQL was more setup and schema management, but in return I got strong relational integrity, joins, and clearer transactional behavior for multi-table operations.

Implementation Journey and Learning Notes:

My first version worked functionally, but I had to think more carefully about how the cart and item tables would support the required access patterns efficiently. The main optimization was using a separate `shopping_cart_items` table with a composite key and indexing `customer_id` in `shopping_carts`, so retrieval by cart ID and customer history queries stayed clean and efficient. A relational design also made transaction handling for add/update item operations more natural. Compared with the Week 5 in-memory version, MySQL added more setup and schema complexity, but it gave persistence across restarts, stronger integrity guarantees, and clearer support for multi-table operations.

```
docker build -t cartapi .
```

```
aws configure
```

```
aws ecr create-repository --repository-name cartapi --region us-east-1
```

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS
```

```
--password-stdin 316009999564.dkr.ecr.us-east-1.amazonaws.com
```

```
docker tag cartapi:latest 316009999564.dkr.ecr.us-east-1.amazonaws.com/cartapi:latest
```

```
docker push 316009999564.dkr.ecr.us-east-1.amazonaws.com/cartapi:latest
```

```
terraform init
```

```
terraform fmt
```

```
terraform validate
```

terraform apply -auto-approve

-var="container_image=316009999564.dkr.ecr.us-east-1.amazonaws.com/cartapi:latest

" -var="db_password=YourStrongPassword123!"

terraform output -raw alb_dns_name

cartapi-alb-1350600380.us-east-1.elb.amazonaws.com

Quick manual test for MySQL

curl -X POST

http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts -H

"Content-Type: application/json" -d

"{\"customer_id\":\"cust-1\",\"customer_email\":\"wilson@example.com\"}"

```
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts -H "Content-Type: application/json" -d "{\"customer_id\":\"cust-1\",\"customer_email\":\"wilson@example.com\"}"
{"cart_id":"db91d4df-0ed5-4ad7-9629-73859f9c6764","customer_id":"cust-1","customer_email":"wilson@example.com","status":"active","created_at":"2026-03-21T22:03:27Z","updated_at":"2026-03-21T22:03:27Z","items":[]}
```

curl -X POST

http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/db91d4df-

0ed5-4ad7-9629-73859f9c6764/items -H "Content-Type: application/json" -d

"{\"product_id\":\"prod-1\",\"name\":\"shirt\",\"quantity\":2,\"unit_price\":19.99}"

```
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/db91d4df-0ed5-4ad7-9629-73859f9c6764/items -H "Content-Type: application/json" -d "{\"product_id\":\"prod-1\",\"name\":\"shirt\",\"quantity\":2,\"unit_price\":19.99}"
{"cart_id":"db91d4df-0ed5-4ad7-9629-73859f9c6764","customer_id":"cust-1","customer_email":"wilson@example.com","status":"active","created_at":"2026-03-21T22:03:27Z","updated_at":"2026-03-21T22:06:04Z","items":[{"product_id":"prod-1","name":"shirt","quantity":2,"unit_price":19.99}]}
```

curl

http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/db91d4df-

0ed5-4ad7-9629-73859f9c6764

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)

\$ curl http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/db91d4df-0ed5-4ad7-9629-73859f9c6764

```
{"cart_id":"db91d4df-0ed5-4ad7-9629-73859f9c6764","customer_id":"cust-1","customer_email":"wilson@example.com","status":"active","created_at":"2026-03-21T22:03:27Z","updated_at":"2026-03-21T22:06:04Z","items":[{"product_id":"prod-1","name":"shirt","quantity":2,"unit_price":19.99}]}
```

MySQL 150-operation test

150 operations, within 5 minutes, saved to mysql_test_results.json, using 50 create, 50 add, and 50 get.

```
HW8 > online-store-cart-api > tests > run_mysql_test.py > ...
8 RESULTS_FILE = "mysql_test_results.json"
9
10 results = []
11 cart_ids = []
12
13 def now_iso():
14     return datetime.now(timezone.utc).isoformat()
15
16 def record(operation, response_time_ms, success, status_code):
17     results.append({
18         "operation": operation,
19         "response_time": round(response_time_ms, 2),
20         "success": success,
21         "status_code": status_code,
22         "timestamp": now_iso()
23     })
```

python run_mysql_test.py

```
HW8 > online-store-cart-api > tests > {} mysql_test_results.json > ...
1 [
2     {
3         "operation": "create_cart",
4         "response_time": 172.66,
5         "success": true,
6         "status_code": 201,
7         "timestamp": "2026-03-21T22:14:28.939881+00:00"
8     },
9     {
10        "operation": "create_cart",
11        "response_time": 56.31,
12        "success": true,
13        "status_code": 201,
14        "timestamp": "2026-03-21T22:14:28.996412+00:00"
15    },
16    {
17        "operation": "create_cart".
```

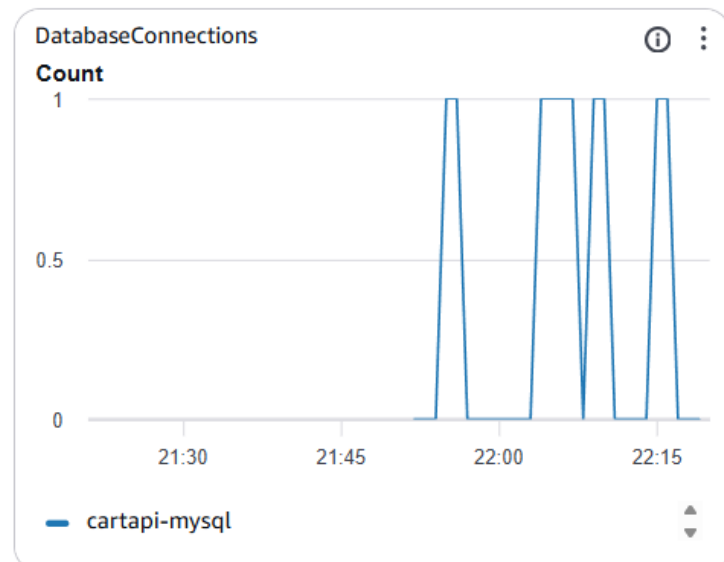
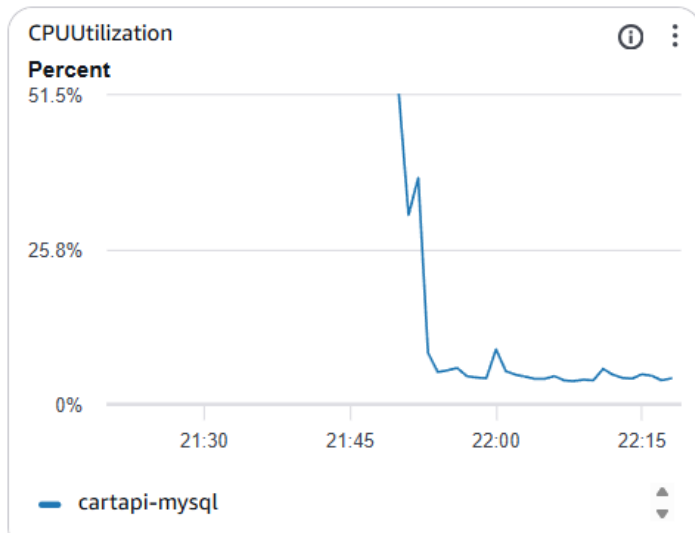
Verify it contains exactly 150 ops

```
python -c "import json; d=json.load(open('mysql_test_results.json')); print(len(d)); from collections import Counter; print(Counter(x['operation'] for x in d))"
```

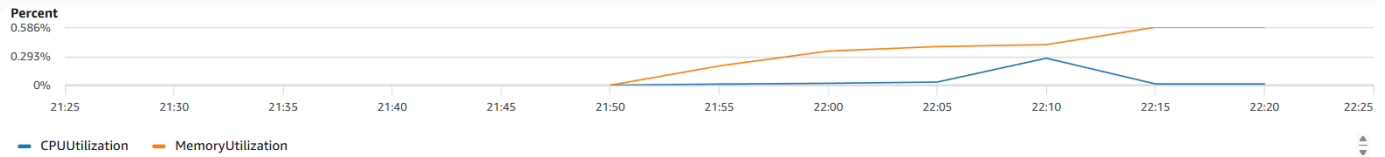
```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW8\online-store-cart-api\tests> python -c
mysql_test_results.json'); print(len(d)); from collections import Counter; print(Counter(x['operation'] for x in d))"
150
Counter({'create_cart': 50, 'add_items': 50, 'get_cart': 50})
```

CloudWatch Monitoring

During the MySQL test, I monitored RDS CPU utilization, database connections, ECS CPU and memory, Application Load Balancer target response time, and RDS read/write metrics in CloudWatch. The screenshots show that the workload stayed relatively small and stable during the 150-operation run. Database connections remained low, which matched my connection pooling configuration and suggested the ECS task was not opening unnecessary sessions. ECS CPU and memory also stayed modest, indicating that this workload was not compute-bound. The RDS read/write latency and IOPS graphs showed short bursts of database activity during test execution, which is expected for create, add-item, and get-cart operations. Overall, the MySQL-backed implementation handled the required test without visible resource stress.



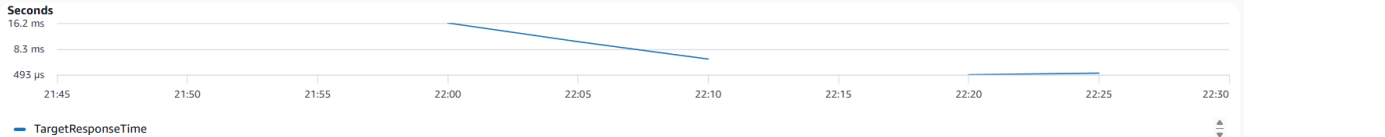
CPUUtilization, MemoryUtilization [🔗](#) 🔄 15 minutes



[Browse \(6\)](#) | [Multi source query](#) | [Graphed metrics \(2\)](#) | [Options](#) | [Source](#) = [Add math](#) [Add query](#)

[All](#) > [ECS](#) > [ClusterName, ServiceName](#) 🔌 Alarm recommendations Download alarm code (2) Create alarm Graph with SQL Graph search

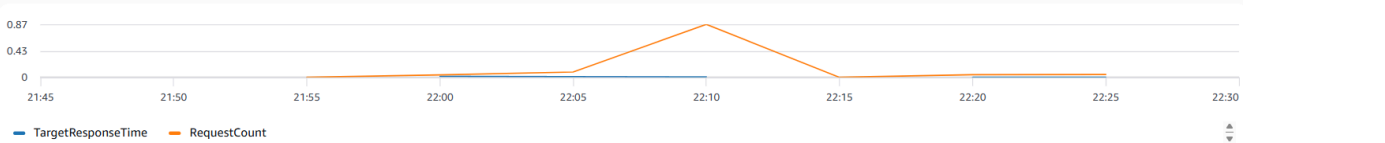
TargetResponseTime [🔗](#) 🔄 15 minutes



[Browse \(4\)](#) | [Multi source query](#) | [Graphed metrics \(1\)](#) | [Options](#) | [Source](#) = [Add math](#) [Add query](#)

[All](#) > [ApplicationELB](#) > [Per AppELB, per AZ, per TG Metrics](#) 🔌 Alarm recommendations Download alarm code Create alarm Graph with SQL Graph search

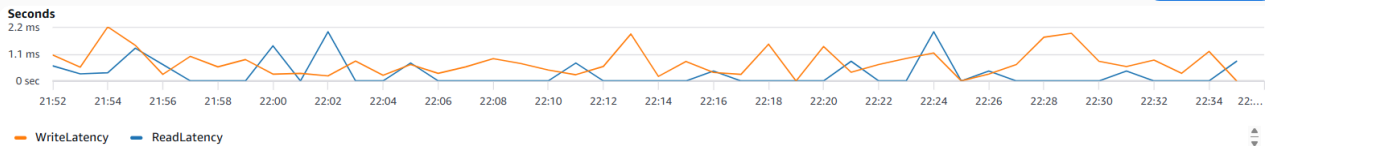
RequestCount, TargetResponseTime [🔗](#) 🔄 15 minutes



[Browse \(4\)](#) | [Multi source query](#) | [Graphed metrics \(2\)](#) | [Options](#) | [Source](#) = [Add math](#) [Add query](#)

[All](#) > [ApplicationELB](#) > [Per AppELB, per AZ, per TG Metrics](#) 🔌 Alarm recommendations Download alarm code Create alarm Graph with SQL Graph search

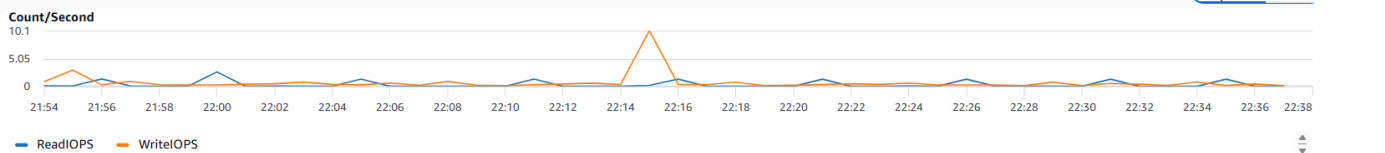
ReadLatency, WriteLatency [🔗](#) 🔄 15 minutes



[Browse \(24\)](#) | [Multi source query](#) | [Graphed metrics \(2\)](#) | [Options](#) | [Source](#) = [Add math](#) [Add query](#)

[All](#) > [RDS](#) > [DBInstanceIdentifier](#) 🔌 Alarm recommendations Download alarm code (2) Create alarm Graph with SQL Graph search

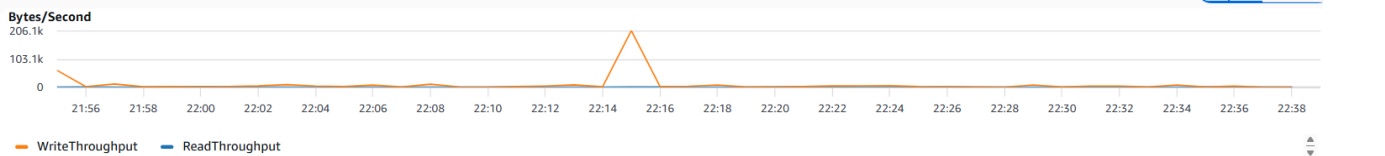
ReadIOPS, WriteIOPS [🔗](#) 🔄 15 minutes



[Browse \(24\)](#) | [Multi source query](#) | [Graphed metrics \(2\)](#) | [Options](#) | [Source](#) = [Add math](#) [Add query](#)

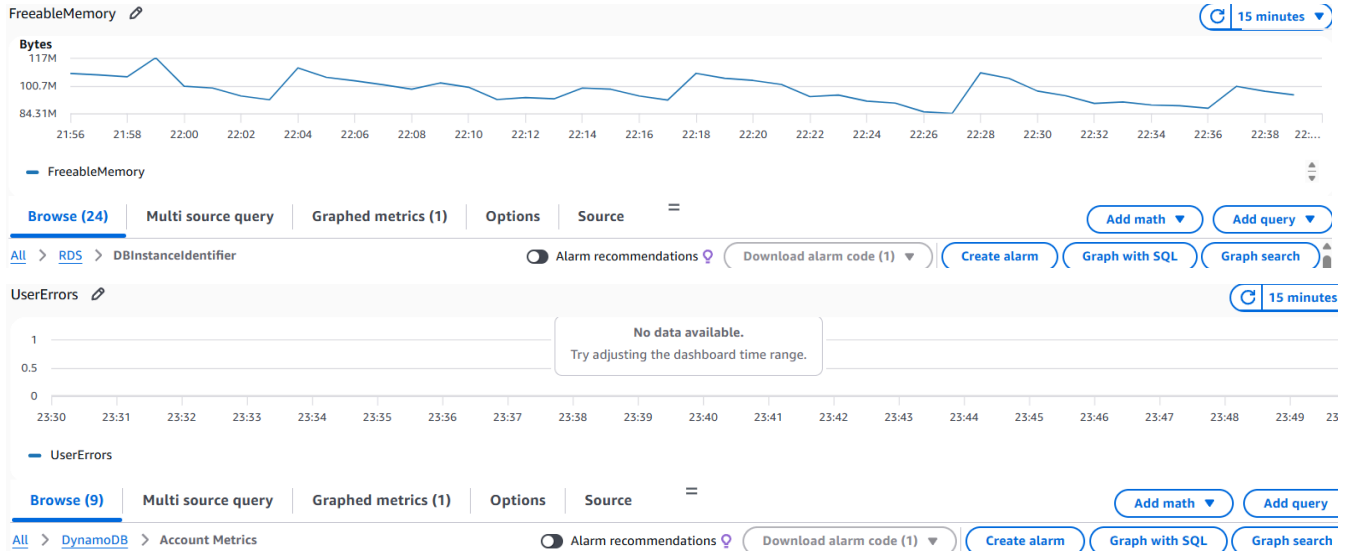
[All](#) > [RDS](#) > [DBInstanceIdentifier](#) 🔌 Alarm recommendations Download alarm code (8) Create alarm Graph with SQL Graph search

ReadThroughput, WriteThroughput [🔗](#) 🔄 15 minutes



[Browse \(24\)](#) | [Multi source query](#) | [Graphed metrics \(2\)](#) | [Options](#) | [Source](#) = [Add math](#) [Add query](#)

[All](#) > [RDS](#) > [DBInstanceIdentifier](#) 🔌 Alarm recommendations Download alarm code (8) Create alarm Graph with SQL Graph search



STEP II: DynamoDB Integration instead of NoSQL

HW8 > online-store-cart-api > terraform >  ecs.tf

```

10 source "aws_ecs_task_definition" "api" {
19   container_definitions = jsonencode([
27     {
28   }
29
30   environment = [
31     { name = "PORT", value = "8080" },
32     { name = "AWS_REGION", value = var.aws_region },
33     { name = "DB_MODE", value = "mysql" },
34     { name = "MYSQL_DSN", value = "${var.db_username}:${var.db_pass}@${var.db_host}/${var.db_name}" },
35     { name = "DYNAMODB_TABLE", value = aws_dynamodb_table.shopping_cart.id }

```

HW8 > online-store-cart-api > terraform >  ecs.tf

```

10 resource "aws_ecs_task_definition" "api" {
19   container_definitions = jsonencode([
27     {
28   }
29
30   environment = [
31     { name = "PORT", value = "8080" },
32     { name = "AWS_REGION", value = var.aws_region },
33     { name = "DB_MODE", value = "dynamodb" },
34     { name = "MYSQL_DSN", value = "" },
35     { name = "DYNAMODB_TABLE", value = aws_dynamodb_table.shopping_cart.id }
36   ]

```

HW8 > online-store-cart-api > src > main.go > CartItem

```
32
33 type ShoppingCart struct {
• 34     CartID          string    `json:"cart_id" dynamodbav:"cart_id"
35     CustomerID      string    `json:"customer_id" dynamodbav:"cus
36     CustomerEmail   string    `json:"customer_email" dynamodbav:"
37     Status          string    `json:"status" dynamodbav:"status"`
38     CreatedAt       string    `json:"created_at" dynamodbav:"crea
39     UpdatedAt       string    `json:"updated_at" dynamodbav:"upda
40     Items           []CartItem `json:"items" dynamodbav:"items"`
41 }
42
43 type CartItem struct {
44     ProductID string `json:"product_id" dynamodbav:"product_id"`
45     Name      string `json:"name" dynamodbav:"name"`
46     Quantity  int    `json:"quantity" dynamodbav:"quantity"`
47     UnitPrice float64 `json:"unit_price" dynamodbav:"unit_price"

```

DynamoDB Design Constraints and Access Patterns

Why you chose your partition key?

I used `cart_id` as the partition key because the main access pattern was retrieving a specific cart by ID. Since cart IDs are UUIDs, they distribute evenly and reduce the chance of hot partitions under normal shopping load.

How you handle the cart-item relationship?

Instead of storing items in a separate table, I embedded the items directly inside the cart record. This made the common access pattern simple because getting a cart also returns all its items with no joins.

Any trade-offs you identified?

The DynamoDB design reduced relational complexity and made reads and writes straightforward, but it introduced NoSQL trade-offs such as eventual consistency and less natural support for relational constraints and multi-entity validation.

Design Evolution and Learning Notes:

My DynamoDB design focused on the main access pattern of retrieving a cart by cart ID and updating it efficiently. I kept the design simple by using `cart_id` as the partition key and embedding items directly inside the cart record. This was different from MySQL, where I used separate tables and relational constraints. The DynamoDB design reduced schema complexity and made the application logic simpler for key-based operations, but it also required thinking more carefully about eventual consistency and the lack of relational joins. In my testing, I validated the design by confirming that the API still supported the same create, add-item, and get-cart workflow as the MySQL version.

```
docker build -t cartapi .
```

```
docker tag cartapi:latest 316009999564.dkr.ecr.us-east-1.amazonaws.com/cartapi:latest
```

```
docker push 316009999564.dkr.ecr.us-east-1.amazonaws.com/cartapi:latest
```

Redeploy

```
terraform apply -auto-approve
```

```
-var="container_image=316009999564.dkr.ecr.us-east-1.amazonaws.com/cartapi:latest
```

```
" -var="db_password=YourStrongPassword123!"
```

```
alb_dns_name = "cartapi-alb-1350600380.us-east-1.elb.amazonaws.com"
```

```
aws ecs update-service \
```

```
--cluster cartapi-cluster \
```

```
--service cartapi \
```

```
--force-new-deployment \
```

```
--region us-east-1
```

```
aws ecs describe-services \
```

```
--cluster cartapi-cluster \
```

```
--services cartapi \
```

```
--region us-east-1
```

Manual test for DynamoDB version

Create Cart:

```
curl -X POST
```

```
http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"customer_id":"cust-d1","customer_email":"ddb@example.com"}'
```

```
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts \
-H "Content-Type: application/json" \
-d '{"customer_id":"cust-d1","customer_email":"ddb@example.com"}'
{"cart_id":"e99501f6-5e82-4009-92c5-d2dcd51f1bfb","customer_id":"cust-d1","customer_email":"ddb@example.com","status":"active","created_at":"2026-03-21T23:15:01Z","updated_at":"2026-03-21T23:15:01Z","items":[]}
```

Add Item:

curl -X POST

```
http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/e99501f6-5e82-4009-92c5-d2dcd51f1bfb/items -H "Content-Type: application/json" -d '{"product_id":"prod-d1","name":"hat","quantity":1,"unit_price":12.99}'
```

```
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/e99501f6-5e82-4009-92c5-d2dcd51f1bfb/items -H "Content-Type: application/json" -d '{"product_id":"prod-d1","name":"hat","quantity":1,"unit_price":12.99}'
{"cart_id":"e99501f6-5e82-4009-92c5-d2dcd51f1bfb","customer_id":"cust-d1","customer_email":"ddb@example.com","status":"active","created_at":"2026-03-21T23:15:01Z","updated_at":"2026-03-21T23:19:13Z","items":[{"product_id":"prod-d1","name":"hat","quantity":1,"unit_price":12.99}]}
```

Get Cart:

curl

```
http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/e99501f6-5e82-4009-92c5-d2dcd51f1bfb
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)

```
$ curl http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/e99501f6-5e82-4009-92c5-d2dcd51f1bfb
{"cart_id":"e99501f6-5e82-4009-92c5-d2dcd51f1bfb","customer_id":"cust-d1","customer_email":"ddb@example.com","status":"active","created_at":"2026-03-21T23:15:01Z","updated_at":"2026-03-21T23:19:13Z","items":[{"product_id":"prod-d1","name":"hat","quantity":1,"unit_price":12.99}]}
```

DynamoDB 150-operation test

HW8 > online-store-cart-api > tests > run_dynamodb_test.py > ...

```
11 cart_ids = []
12
13 def now_iso():
14     return datetime.now(timezone.utc).isoformat()
15
16 def record(operation, response_time_ms, success, status_code):
17     results.append({
18         "operation": operation,
19         "response_time": round(response_time_ms, 2),
20         "success": success,
21         "status_code": status_code,
22         "timestamp": now_iso()
23     })
```

python run_dynamodb_test.py

HW8 > online-store-cart-api > tests > {} dynamodb_test_results.json > ...

```
1  [
2    {
3      "operation": "create_cart",
4      "response_time": 191.39,
5      "success": true,
6      "status_code": 201,
7      "timestamp": "2026-03-21T23:24:57.065380+00:00"
8    },
9    {
10     "operation": "create_cart",
11     "response_time": 58.8,
12     "success": true,
13     "status_code": 201,
14     "timestamp": "2026-03-21T23:24:57.123994+00:00"
15   },
16   {
17     "operation": "create_cart",
18     "response_time": 59.14,
19     "success": true,
```

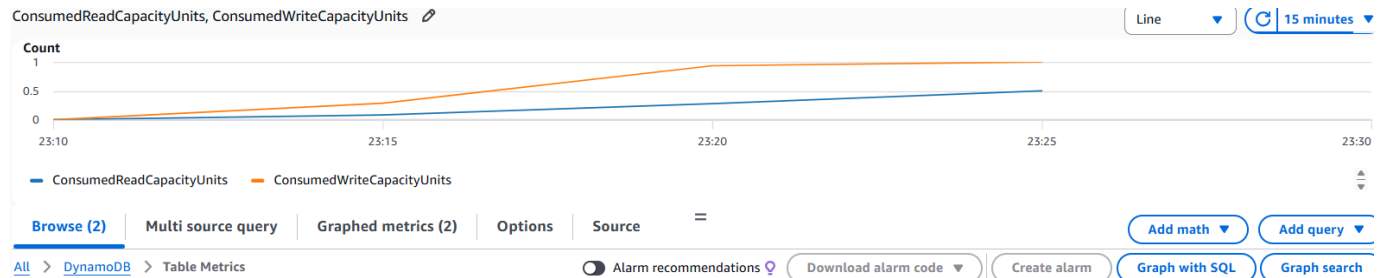
Verify

```
python -c "import json; d=json.load(open('dynamodb_test_results.json')); print(len(d));
from collections import Counter; print(Counter(x['operation'] for x in d))"
```

PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW8\online-store-cart-api\tests> python -c "import json; d=json.load(open('dynamodb_test_results.json')); print(len(d)); from collections import Counter; print(Counter(x['operation'] for x in d))"

150
Counter({'create_cart': 50, 'add_items': 50, 'get_cart': 50})





Eventual consistency investigation for DynamoDB

alb_dns_name = "cartapi-alb-1350600380.us-east-1.elb.amazonaws.com"

Test 1 — Create then immediate GET

curl -X POST

http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts -H

"Content-Type: application/json" -d

"{"customer_id":"ec-1","customer_email":"ec1@example.com"}"

curl

http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f

```
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts -H "Content-Type: application/json" -d '{"customer_id":"ec-1","customer_email":"ec1@example.com"}'
{"cart_id":"6de11ffb-eaad-431e-a376-c8503e8a0f6f","customer_id":"ec-1","customer_email":"ec1@example.com","status":"active","created_at":"2026-03-21T23:42:38Z","updated_at":"2026-03-21T23:42:38Z","items":[]}
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)

```
$ curl http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f
{"cart_id":"6de11ffb-eaad-431e-a376-c8503e8a0f6f","customer_id":"ec-1","customer_email":"ec1@example.com","status":"active","created_at":"2026-03-21T23:42:38Z","updated_at":"2026-03-21T23:42:38Z","items":[]}
```

Test 2 — Add item then immediate GET

curl -X POST

http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items -H "Content-Type: application/json" -d

"{"product_id":"rapid-1","name":"rapid-item","quantity":1,"unit_price":3.99}"

curl

http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f

```
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items -H "Content-Type: application/json" -d '{"product_id":"rapid-1","name":"rapid-item","quantity":1,"unit_price":3.99}'
{"cart_id":"6de11ffb-eaad-431e-a376-c8503e8a0f6f","customer_id":"ec-1","customer_email":"ec1@example.com","status":"active","created_at":"2026-03-21T23:42:38Z","updated_at":"2026-03-21T23:46:59Z","items":[{"product_id":"rapid-1","name":"rapid-item","quantity":1,"unit_price":3.99}]}
```

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)

```
$ curl http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f
{"cart_id":"6de11ffb-eaad-431e-a376-c8503e8a0f6f","customer_id":"ec-1","customer_email":"ec1@example.com","status":"active","created_at":"2026-03-21T23:42:38Z","updated_at":"2026-03-21T23:46:59Z","items":[{"product_id":"rapid-1","name":"rapid-item","quantity":1,"unit_price":3.99}]}
```


Test 3 — Rapid update same cart

curl -X POST

```
http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items -H "Content-Type: application/json" -d '{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}'
```

```
Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)
$ http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items -H "Content-Type: application/json" -d '{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}'
bash: http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items: No such file or directory

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items -H "Content-Type: application/json" -d '{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}'
{"cart_id":"6de11ffb-eaad-431e-a376-c8503e8a0f6f","customer_id":"ec-1","customer_email":"ec1@example.com","status":"active","created_at":"2026-03-21T23:42:38Z","updated_at":"2026-03-21T23:51:38Z","items":[{"product_id":"rapid-1","name":"rapid-item","quantity":1,"unit_price":3.99},{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}]

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items -H "Content-Type: application/json" -d '{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}'
{"cart_id":"6de11ffb-eaad-431e-a376-c8503e8a0f6f","customer_id":"ec-1","customer_email":"ec1@example.com","status":"active","created_at":"2026-03-21T23:42:38Z","updated_at":"2026-03-21T23:51:43Z","items":[{"product_id":"rapid-1","name":"rapid-item","quantity":1,"unit_price":3.99},{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}]

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items -H "Content-Type: application/json" -d '{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}'
{"cart_id":"6de11ffb-eaad-431e-a376-c8503e8a0f6f","customer_id":"ec-1","customer_email":"ec1@example.com","status":"active","created_at":"2026-03-21T23:42:38Z","updated_at":"2026-03-21T23:51:46Z","items":[{"product_id":"rapid-1","name":"rapid-item","quantity":1,"unit_price":3.99},{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}]

Owner@DESKTOP-5FCT03U MINGW64 ~/Documents/Audacity/Building-Scalable-Distributed-Systems (main)
$ curl -X POST http://cartapi-alb-1350600380.us-east-1.elb.amazonaws.com/shopping-carts/6de11ffb-eaad-431e-a376-c8503e8a0f6f/items -H "Content-Type: application/json" -d '{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}'
{"cart_id":"6de11ffb-eaad-431e-a376-c8503e8a0f6f","customer_id":"ec-1","customer_email":"ec1@example.com","status":"active","created_at":"2026-03-21T23:42:38Z","updated_at":"2026-03-21T23:51:48Z","items":[{"product_id":"rapid-1","name":"rapid-item","quantity":1,"unit_price":3.99},{"product_id":"rapid-2","name":"rapid-item-2","quantity":2,"unit_price":7.99}]
```

How frequently do you observe eventual consistency delays? What application patterns are most affected by consistency delays? How can you design your application to handle consistency gracefully?

In my small test runs, I did not consistently observe obvious read-after-write delays, so eventual consistency was not very visible at this scale. The application pattern most affected would be rapid read-after-write behavior, especially when a user adds an item and immediately expects every client view to reflect that update. For a shopping cart system, the safest design is to tolerate short-lived stale reads, refresh after writes when needed, or use stronger consistency selectively for the most user-visible operations.

STEP III: Database Comparison & Analysis

```
HW8 > online-store-cart-api > tests > merge_results.py > ...
1 import json
2
3 with open("mysql_test_results.json", "r", encoding="utf-8") as f:
4     mysql_data = json.load(f)
5
6 with open("dynamodb_test_results.json", "r", encoding="utf-8") as f:
7     dynamodb_data = json.load(f)
8
9 assert len(mysql_data) == 150, f"MySQL count is {len(mysql_data)}"
10 assert len(dynamodb_data) == 150, f"DynamoDB count is {len(dynamo
11
12 combined = {
13     "mysql": mysql_data,
14     "dynamodb": dynamodb_data
15 }
16
17 with open("combined_results.json", "w", encoding="utf-8") as f:
18     json.dump(combined, f, indent=2)
19
20 print("combined_results.json created")
```

python merge_results.py

Pre-Analysis Data Verification:

I merged mysql_test_results.json and dynamodb_test_results.json into combined_results.json and used that single combined file as the source for all comparison analysis. I also verified that each dataset contained exactly 150 operations: 50 create_cart, 50 add_items, and 50 get_cart, so the comparison would be fair and consistent.

HW8 > online-store-cart-api > tests > {} combined_results.json > ...

```
1 {
2   "mysql": [
3     {
4       "operation": "create_cart",
5       "response_time": 172.66,
6       "success": true,
7       "status_code": 201,
8       "timestamp": "2026-03-21T22:14:28.939881+00:00"
9     },
10    {
11      "operation": "create_cart",
12      "response_time": 56.31,
13      "success": true,
14      "status_code": 201,
15      "timestamp": "2026-03-21T22:14:28.996412+00:00"
16    },
17    {
18      "operation": "create_cart",
19      "response_time": 60.94,
```

Verify

```
python -c "import json; d=json.load(open('combined_results.json')); print(len(d['mysql']), len(d['dynamodb']))"
```

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-S:
combined_results.json'); print(len(d['mysql']), len(d['dynamodb']))"
150 150
```

HW8 > online-store-cart-api > tests > analyze_results.py > ...

```
15 def summarize(data):
16     times = [x["response_time"] for x in data]
17     success_rate = (sum(1 for x in data if x["success"])) / len(da
18     return {
19         "avg": round(statistics.mean(times), 2),
20         "p50": round(percentile(times, 50), 2),
21         "p95": round(percentile(times, 95), 2),
22         "p99": round(percentile(times, 99), 2),
23         "success_rate": round(success_rate, 2),
24         "total": len(data),
25     }
26
27 def summarize_op(data, op):
28     subset = [x["response_time"] for x in data if x["operation"]
29     return round(statistics.mean(subset), 2)
30
31 with open("combined_results.json", "r", encoding="utf-8") as f:
32     combined = json.load(f)
```

python analyze_results.py

```
PS C:\Users\Owner\Documents\Audacity\Building-Scalable-Distributed-Systems\HW8\online-store-cart-api\te
MYSQL {'avg': 57.39, 'p50': 55.44, 'p95': 73.87, 'p99': 93.0, 'success_rate': 100.0, 'total': 150}
DYNAMODB {'avg': 60.25, 'p50': 58.22, 'p95': 75.88, 'p99': 112.79, 'success_rate': 100.0, 'total': 150}
create_cart mysql 58.98 dynamodb 61.21
add_items mysql 59.3 dynamodb 63.42
get_cart mysql 53.89 dynamodb 56.13
```

Performance Comparison Table

Metric	MySQL	DynamoDB	Winner	Margin
Avg Response Time (ms)	57.39	60.25	MySQL	2.86 ms
P50 Response Time (ms)	55.44	58.22	MySQL	2.78 ms
P95 Response Time (ms)	73.87	75.88	MySQL	2.01 ms
P99 Response Time (ms)	93.00	112.79	MySQL	19.79 ms
Success Rate (%)	100.0	100.0	Tie	0.0
Total Operations	150	150	Tie	0

Performance Comparison Table

Operation	MySQL Avg (ms)	DynamoDB Avg (ms)	Faster By
CREATE_CAR T	58.98	61.21	2.23 ms
ADD_ITEMS	59.30	63.42	4.12 ms
GET_CART	53.89	56.13	2.24 ms

Looking at the comparison results, MySQL was consistently faster than DynamoDB in this assignment's shopping cart workload. The difference was not huge for average latency, but MySQL still won across every major latency metric and every operation type. The biggest gap showed up at p99, where MySQL had 93.00 ms and DynamoDB had 112.79 ms, which suggests MySQL also had better worst-case tail performance in this test. Since both databases had a 100% success rate, the decision came down more to latency, data model fit, and consistency than basic correctness.

Consistency Model Impact Assessment

MySQL gave stronger ACID-style consistency for relational updates, especially when working across `shopping_carts` and `shopping_cart_items` in a transaction. DynamoDB simplified the data model by embedding items, but default reads are eventually consistent. In my own consistency checks, I did not consistently observe obvious read-after-write delays at this scale, but the DynamoDB model still requires the application to tolerate short-lived stale reads in principle. In practice, this means MySQL is easier for workloads where immediate correctness across related records matters, while DynamoDB is easier for scale-oriented key-value style access patterns.

Resource efficiency analysis

MySQL required more operational overhead because I had to manage an RDS instance, configure connection pooling, design a relational schema, and think about transactions and joins. DynamoDB reduced some of that operational complexity because it is fully managed and the application did not need SQL connection pooling or relational schema management. From an implementation perspective, DynamoDB was simpler for key-based cart operations, while MySQL gave me more explicit control over relational integrity and transaction behavior. In terms of resource patterns, MySQL involved more infrastructure planning, while DynamoDB shifted more of that burden to the managed service model.

Real-World Scenario Recommendations

Scenario A: Startup MVP (100 users/day, 1 developer, limited budget, quick launch)

I would recommend DynamoDB since it was simpler to operate, required less schema management, and still supported the same cart API. For a very small team, lower operational complexity matters a lot.

Scenario B: Growing Business (10K users/day, 5 developers, moderate budget, feature expansion)

I would recommend MySQL since it provides stronger relational structure, clearer support for customer history and related entities, and better fit if the shopping cart begins to interact more tightly with other relational business data.

Scenario C: High-Traffic Events (50K normal, 1M spike users, revenue-critical, can invest in infrastructure)

I would lean toward DynamoDB since it is a better fit for very high-scale key-based access patterns and reduces the burden of managing connections and relational scaling during spikes.

Scenario D: Global Platform (millions of users, multi-region, 24/7 availability, enterprise requirements)

I would recommend DynamoDB for globally distributed, since it is always-on key-value workloads, while MySQL remains useful where relational consistency and reporting matter more.

Your Evidence-Based Architecture Recommendations

Shopping Cart Winner: Which database would you choose for shopping carts? Why?

Based on my results, I would choose MySQL for shopping carts because it gave me the better overall combination of response time, consistency, and fit for this exact API. MySQL had the lower average response time at 57.39 ms compared with 60.25 ms for DynamoDB, so it had an average advantage of 2.86 ms. It was also faster at the p50, p95, and p99 levels, and it was faster in all three main shopping cart operations: `create_cart`, `add_items`, and `get_cart`. On top of that, MySQL fits this shopping cart design well because the cart and cart items naturally have a relational structure, and MySQL gives stronger transactional consistency for updates across related records. Even though DynamoDB was simpler operationally, MySQL ended up being the better match for this specific shopping cart workload.

Your Polyglot Strategy: If building a complete e-commerce system, how would you use both?

If I were building a complete e-commerce system, I would use a polyglot approach. I would store shopping carts in MySQL based on my test results, because MySQL performed better for this API and also matched the relational structure of carts and cart items. I would store user sessions in DynamoDB because they are key-based, high-volume, and a good fit for a simpler NoSQL access pattern. I would use DynamoDB or a cache-backed store for the product catalog if the workload is mostly read-heavy and needs fast lookups at scale. I would use MySQL for order history because orders are more relational, transactional, and reporting-oriented. So for a full system, I would not force one database to do everything. I would use each one where its strengths make the most sense.

Learning Reflection

What Surprised Me:

What surprised me most was that both implementations could support the same shopping cart API while requiring very different design thinking. MySQL made relationships and constraints more explicit, while DynamoDB made the data model simpler for cart retrieval and updates.

What Failed Initially:

The main challenge was not the API itself, but adapting the storage model correctly for each database. MySQL required more thought around schema design, transactions, and connection management. DynamoDB required a different mindset around embedded items, partition keys, and eventual consistency.

Key Insights Gained:

I would definitely choose MySQL when relational integrity, transactions, and stronger consistency matter most. I would definitely choose DynamoDB when the workload is primarily key-based, high-scale, and operational simplicity is important. The biggest lesson from implementing both was that database choice is not just about speed. It is also about access patterns, data relationships, consistency needs, and operational complexity.

What I Would Tell Another Student:

I would tell another student to focus first on the actual access patterns before choosing a schema or table design. This assignment made the trade-offs much clearer through hands-on implementation than they would have been from theory alone.

terraform destroy -auto-approve

-var="container_image=316009999564.dkr.ecr.us-east-1.amazonaws.com/cartapi:latest"
-var="db_password=YourStrongPassword123!"